অধ্যায় ১৫

JavaScript Browser APIs, Web Storage, Fetch, URL এবং Real-World Browser Applications

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- JavaScript ভাষা এবং Browser-এর দেওয়া API-র পার্থক্য করতে, এবং Feature Detection করতে।
- `URL` ও `URLSearchParams` দিয়ে ঠিকানা নিরাপদে পড়তে, গড়তে ও যাচাই করতে।
- `localStorage` ও `sessionStorage`-এর পার্থক্য, সীমা ও ত্রুটি সামলাতে, এবং `storage` Event দিয়ে একাধিক Tab সমন্বয় করতে।
- সংরক্ষিত তথ্যে Version রেখে পুরোনো তথ্য নতুন গঠনে রূপান্তর (Migration) করতে।
- `Request`, `Response` ও `fetch` দিয়ে JSON পাঠাতে ও আনতে।
- History API দিয়ে Page-এর অবস্থা URL-এ রাখতে, যাতে Link শেয়ার ও Reload-এ অবস্থা টিকে থাকে।
- `Blob` ও `File` দিয়ে File তৈরি, Download এবং ব্যবহারকারীর দেওয়া File পড়তে।
- Browser-এ তথ্য রাখার নিরাপত্তা ও গোপনীয়তার সীমা বুঝতে।
- একটি সম্পূর্ণ **Bookmark Vault** তৈরি করতে এবং আসল Browser-এ তার প্রতিটি API পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৯: DOM, Event Delegation, `localStorage`-এর প্রাথমিক ব্যবহার।
- অধ্যায় ১০: `fetch`, Promise, `AbortController`।
- অধ্যায় ১১: প্রত্যাশিত ত্রুটি ও Trust Boundary।
- অধ্যায় ১২: ES Module ও Local Server।

এই অধ্যায় কেন গুরুত্বপূর্ণ

JavaScript ভাষাটি নিজে শুধু হিসাব, তথ্য ও নিয়ন্ত্রণ জানে; Page দেখানো, ঠিকানা পড়া, তথ্য সংরক্ষণ বা File নামানো—এর কোনোটিই ভাষার অংশ নয়। এগুলো দেয় Browser, তার API-র মাধ্যমে। একটি বাস্তব Web Application-এর বেশির ভাগ কাজই এই API-গুলোর ব্যবহার: খোঁজার শব্দ URL-এ রাখা যাতে Link শেয়ার করা যায়, ব্যবহারকারীর পছন্দ মনে রাখা, অর্ধেক লেখা Form হারিয়ে না যাওয়া, তথ্য File হিসেবে নামানো ও আবার তোলা।

প্রতিটি API-র নিজস্ব সীমা ও ফাঁদ আছে: Storage ভর্তি হয়ে যায়, URL-এ বিপজ্জনক Protocol লুকিয়ে থাকে, ব্যবহারকারী যেকোনো File দিতে পারেন। এই অধ্যায়ে আপনি API-গুলো শিখবেন তাদের ত্রুটির পথসহ, এবং Project-এ প্রতিটি API আসল Browser-এ পরীক্ষিত হবে।

১৫.১ ভাষা এবং পরিবেশ

একই JavaScript ভাষা Browser ও Node.js দুই জায়গায় চলে, কিন্তু প্রতিটি পরিবেশ নিজের API যোগ করে। Browser-এ দেখুন:

browser-env.js · JavaScript — চলবে: Browser

```
console.log(typeof window, typeof document, location.protocol);
console.log('localStorage' in window, isSecureContext);
console.log(typeof URL, typeof fetch);
```

Output

```
object object http:
true true
function function
```

একই প্রশ্ন Node.js-এ:

node-env.js · JavaScript — চলবে: Node.js

```
console.log(typeof window, typeof document, typeof process);
console.log(typeof URL, typeof fetch);
```

Output

```
undefined undefined object
function function
```

`window`, `document`, `location`, `localStorage` শুধু Browser-এর; `process` শুধু Node.js-এর। `URL` ও `fetch` দুই জায়গাতেই আছে, কারণ উভয় পরিবেশ একই মানদণ্ড অনুসরণ করে। তাই একটি Module-এ শুধু `URL`, `fetch`, `JSON`-এর মতো সাধারণ API ব্যবহার করলে সেটি দুই জায়গাতেই চলে এবং Node.js-এ Test করা যায় (অধ্যায়

১২-এর Domain স্তর)।

Feature Detection: একটি API ব্যবহারের আগে দেখে নিন সেটি আছে কি না, যেমন `if ('share' in navigator)`। Browser-এর নাম বা সংস্করণ দেখে অনুমান করবেন না; একই Browser-এর ভিন্ন সংস্করণে, বা নিরাপত্তা-সেটিংসে, API থাকতে বা না থাকতে পারে। কিছু API শুধু **Secure Context**-এ পাওয়া যায়, অর্থাৎ `https://` বা `localhost`-এ; `isSecureContext` তা জানায়।

১৫.২ URL এবং URLSearchParams

URL হাতে লেখা String জোড়া দিয়ে বানানো বা ভাঙা ভুলের উৎস: `?` আর `&` গুলিয়ে যায়, বিশেষ অক্ষর Encode হয় না। `URL` Object এ কাজ নির্ভুলভাবে করে।

url.js · JavaScript — চলবে: Browser Console ও Node.js

```
const address =
  'https://shop.example.com:8080/products/list?category=books&page=2#top';
const url = new URL(address);

console.log(url.hostname, url.port, url.pathname);
const { searchParams } = url;
console.log(searchParams.get('category'), searchParams.get('page'), url.hash);

url.searchParams.set('page', '3');
url.searchParams.append('tag', 'বই');
console.log(url.search);

const relative = new URL('../about', 'https://example.com/docs/intro/');
console.log(relative.href);

try {
  new URL('not a url');
} catch (error) {
  console.log(error.name);
}
```

Output

```
shop.example.com 8080 /products/list
books 2 #top
?category=books&page=3&tag=%E0%A6%AC%E0%A6%87
https://example.com/docs/about
TypeError
```

- **অংশগুলো আলাদা Property-তে:** `protocol`, `hostname`, `port`, `pathname`, `search`, `hash`।

- `searchParams` Query অংশ পড়ে ও লেখে; `set` বদলায়, `append` যোগ করে। বাংলা `বই` নিজে থেকেই Encode হয়েছে (`%E0%A6%AC...`)। `get` করলে আবার `বই` পাওয়া যায়।
- **আপেক্ষিক ঠিকানা:** দ্বিতীয় Argument একটি ভিত্তি; `../about` তার সঙ্গে মিলিয়ে পূর্ণ ঠিকানা হয়েছে।
- **অবৈধ লেখা `TypeError` দেয়,** তাই `new URL()` নিজেই একটি যাচাইয়ের উপায়, যা Project ব্যবহার করবে।

`URLSearchParams` আলাদাভাবেও ব্যবহার করা যায়, যেমন একটি Object থেকে Query তৈরি:

search-params.js · JavaScript — চলবে: Browser Console ও Node.js

```
const params = new URLSearchParams({ q: 'js & dom', sort: 'new' });
console.log(params.toString());

for (const [key, value] of params) {
  console.log(`${key} = ${value}`);
}

const fromAddress = new URLSearchParams('?page=abc&size=20');
const page = Number(fromAddress.get('page'));
console.log(Number.isInteger(page) ? page : 1, fromAddress.get('missing'));
```

Output

```
q=js+%26+dom&sort=new
q = js & dom
sort = new
1 null
```

`&` চিহ্নটি Encode হয়েছে, নইলে Server সেটিকে একটি নতুন Parameter-এর শুরু ভাবত। শেষ লাইনে দেখুন: URL-এর মান সব সময় লেখা, এবং ব্যবহারকারী যেকোনো কিছু লিখতে পারেন (`page=abc`), তাই URL-ও একটি Trust Boundary: রূপান্তরের পর যাচাই, অবৈধ হলে নিরাপদ Default।

> **সতর্কতা:** ব্যবহারকারীর দেওয়া URL Link হিসেবে বসানোর আগে Protocol যাচাই করুন। `javascript:alert(1)` একটি বৈধ URL; `<a href>` -এ বসালে Click-এ Code চলে। শুধু `http:` ও `https:` অনুমোদন করুন।

১৫.৩ Web Storage: localStorage এবং sessionStorage

অধ্যায় ৯-এ `localStorage` ব্যবহার করেছেন। এর একটি সঙ্গী আছে: `sessionStorage`। দুটির ব্যবহারের নিয়ম একই, পার্থক্য তাদের আয়ুতে।

storage-kinds.js · JavaScript চলবে: Browser

```
localStorage.setItem('theme', 'dark');
sessionStorage.setItem('draft', 'অর্ধেক লেখা');

console.log(localStorage.getItem('theme'), sessionStorage.getItem('draft'));
console.log(sessionStorage.length, typeof sessionStorage.key(0));

localStorage.removeItem('theme');
sessionStorage.clear();
console.log(localStorage.getItem('theme'), sessionStorage.length);
```

Output

```
dark অর্ধেক লেখা
1 string
null 0
```

	localStorage	sessionStorage
কতদিন থাকে	মুছে না ফেলা পর্যন্ত	Tab বন্ধ হওয়া পর্যন্ত (Reload-এ থাকে)
কে দেখে	একই Website-এর সব Tab	শুধু সেই Tab
উপযুক্ত ব্যবহার	পছন্দ, সংরক্ষিত তালিকা	অর্ধেক পূরণ করা Form, Tab-এর সাময়িক অবস্থা
সীমা	সাধারণত প্রতি Website প্রায় ৫ MB	একই

দুটিই Synchronous (কাজ শেষ না হওয়া পর্যন্ত Program থামে), শুধু লেখা রাখে, এবং ব্যর্থ হতে পারে। জায়গা শেষ হলে কী হয়, আসল Browser-এ দেখুন:

quota.js · JavaScript চলবে: Browser

```
const huge = 'x'.repeat(11 * 1024 * 1024);

try {
  localStorage.setItem('huge', huge);
  console.log('সংরক্ষিত');
} catch (error) {
  console.log(`ব্যর্থ: ${error.name}`);
}
console.log(localStorage.getItem('huge'));
```

Output

```
ব্যর্থ: QuotaExceededError
null
```

প্রায় এক কোটি ১৫ লক্ষ অক্ষরের লেখা সীমা ছাড়িয়েছে; `setItem` একটি `QuotaExceededError` ছুড়েছে এবং কিছুই সংরক্ষিত হয়নি। Private Browsing-এ বা ব্যবহারকারী Storage বন্ধ রাখলেও একই ধরনের ত্রুটি আসতে পারে। তাই প্রতিটি `setItem` একটি `try` -এর ভেতরে, এবং ব্যর্থ হলে ব্যবহারকারীকে জানানো (অধ্যায় ৯-এর Task Manager)।

কখন Web Storage যথেষ্ট নয়

Web Storage ছোট, সরল তথ্যের জন্য: পছন্দ, কয়েকশো সারির তালিকা। তিনটি পরিস্থিতিতে অন্য ব্যবস্থা লাগে। **অনেক তথ্য** (হাজারো সারি, ছবি, File): সীমা কয়েক MB, আর বড় JSON প্রতিবার পুরোটা পড়া ও লেখা ধীর। **খোঁজা ও সূচি** (Index) দরকার হলে: Web Storage শুধু Key দিয়ে খোঁজে। **Page আটকানো চলবে না**: Web Storage Synchronous, তাই বড় লেখার সময় Page থেমে থাকে। এসব ক্ষেত্রে Browser-এর **IndexedDB** (Asynchronous, বড়, সূচিযুক্ত Database) এবং Offline-এ ব্যবহারের জন্য Network-এর সাড়া রাখতে **Cache API** ব্যবহৃত হয়। দুটিরই API তুলনামূলক জটিল; বাস্তবে অনেকেই এগুলোর ওপর তৈরি ছোট Library ব্যবহার করেন। এই বইয়ের Project-গুলোর তথ্য ছোট, তাই Web Storage-ই উপযুক্ত।

storage Event: একাধিক Tab-এর সমন্বয়

একই Website দুটি Tab-এ খোলা থাকলে, একটিতে `localStorage` বদলালে **অন্যটি** একটি `storage` Event পায়। নিচের Page-এ একটি `<iframe>` -এ একই Website-এর আরেকটি Page আছে, যা দ্বিতীয় Tab-এর মতো আচরণ করে:

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>storage Event</title>
</head>
<body>
  <iframe id="other" src="other.html" title="অন্য Tab-এর মতো"></iframe>
  <script src="main.js"></script>
</body>
</html>
```

other.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>অন্য Page</title>
</head>
<body>
  <script>
    window.addEventListener('storage', (event) => {
      console.log(`অন্য Page দেখল: ${event.key} = ${event.newValue}`);
    });
  </script>
</body>
</html>
```

main.js · JavaScript চলবে: Browser

```javascript
let ownEvents = 0;
window.addEventListener('storage', () => {
  ownEvents += 1;
});

document.querySelector('#other').addEventListener('load', () => {
  localStorage.setItem('last-update', '10:30');
  setTimeout(() => {
    console.log(`এই Page নিজে Event পেয়েছে ${ownEvents} বার`);
    localStorage.removeItem('last-update');
  }, 200);
});
```

Output

```
অন্য Page দেখল: last-update = 10:30
এই Page নিজে Event পেয়েছে 0 বার
অন্য Page দেখল: last-update = null
```

যে Page পরিবর্তন করেছে সে নিজে Event পায় না; পায় একই Website-এর অন্য সব Page। শেষ লাইনটিও লক্ষ করুন: `removeItem`-ও একটি Event পাঠায়, যার `newValue` `null`। তাই Listener-কে মুছে ফেলার ক্ষেত্রটিও সামলাতে হয়। Event Object-এ থাকে কোন Key (`key`), আগের মান (`oldValue`) এবং নতুন মান (`newValue`)। Project এই Event দিয়ে দুটি Tab-এর তালিকা একই রাখে। `sessionStorage`-এর জন্য এই Event অন্য Tab-এ যায় না, কারণ প্রতিটি Tab-এর `sessionStorage` আলাদা।

১৫.৪ সংরক্ষিত তথ্যের Version এবং Migration

আজ আপনি কাজের তালিকা রাখছেন লেখার Array হিসেবে। পরের মাসে প্রতিটি কাজের সঙ্গে "সম্পন্ন" অবস্থা রাখতে চাইলেন। ব্যবহারকারীদের Browser-এ পুরোনো গঠনের তথ্য রয়ে গেছে; নতুন Code সেটি পড়তে না পারলে তাদের তালিকা হারাবে। সমাধান: সংরক্ষিত তথ্যে একটি `version` রাখা, এবং পড়ার সময় পুরোনো Version-কে নতুনে রূপান্তর করা।

migrate.js · JavaScript — চলবে: Browser Console ও Node.js

```
function migrate(data) {
  if (data?.version === 1) {
    const items = data.items.map((name) => ({ name, done: false }));
    return { version: 2, items };
  }
  if (data?.version === 2) {
    return data;
  }
  throw new Error(`অচেনা Version: ${data?.version}`);
}

const saved = { version: 1, items: ['বাজার', 'বই পড়া'] };
const current = migrate(saved);

console.log(current.version, current.items[0].name, current.items[0].done);
console.log(migrate(current) === current);

try {
  migrate({ version: 9 });
} catch (error) {
  console.log(error.message);
}
```

Output

```
2 বাজার false
true
অচেনা Version: 9
```

প্রতিটি পুরোনো Version-এর জন্য একটি ধাপ, এবং ধাপগুলো ক্রমে চলে (১ থেকে ২, ২ থেকে ৩...)। অচেনা Version (হয়তো নতুন সংস্করণের App থেকে, বা ভাঙা) চুপচাপ গ্রহণ না করে স্পষ্ট ত্রুটি। Project-এর Export File-এও `version` আছে, একই কারণে।

১৫.৫ fetch, Request এবং Response

অধ্যায় ১০-এ `fetch` দিয়ে তথ্য এনেছেন। Server-এ তথ্য **পাঠাতে** Method, Header ও Body দিতে হয়। `fetch`-এর ভেতরে এগুলো একটি `Request` Object-এ জমা হয়; সরাসরি `Request` তৈরি করে দেখা যায় কী পাঠানো হবে, Network ছাড়াই:

request.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const request = new Request('https://api.example.com/bookmarks', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title: 'MDN', url: 'https://developer.mozilla.org/' }),
});

console.log(request.method, request.headers.get('content-type'));
console.log((await request.json()).title);

const response = new Response(JSON.stringify({ id: 7 }), {
  status: 201,
  headers: { 'Content-Type': 'application/json' },
});
console.log(response.status, response.ok, (await response.json()).id);
```

Output

```
POST application/json
MDN
201 true 7
```

- `method: 'POST'` তথ্য পাঠানোর জন্য; `GET` (Default) আনার জন্য।
- **Body লেখা হিসেবে যায়;** Object পাঠাতে `JSON.stringify`, এবং `Content-Type` Header দিয়ে Server-কে জানাতে হয় এটি JSON।
- **Header-এর নাম বড়-ছোট হাতের অক্ষর নির্বিশেষে;** `get('content-type')` কাজ করেছে।
- `fetch(url, options)` লিখলে ঠিক এই `Request`-টিই পাঠানো হয়। অধ্যায় ১০-এর কৃত্রিম `Response` দিয়ে Server-এর সাড়াও একইভাবে তৈরি করা যায়।

একই Website-এর একটি JSON File আনা, আসল Browser-এ:

links.json · JSON — চলবে: Browser

```
[
  { "title": "MDN", "url": "https://developer.mozilla.org/" },
  { "title": "Node.js", "url": "https://nodejs.org/" }
]
```

index.html · HTML চলবে: Browser

```html
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <title>fetch</title>
  <script type="module" src="app.mjs"></script>
</head>
<body></body>
</html>
```

app.mjs · JavaScript চলবে: Browser

```javascript
const response = await fetch('links.json');
console.log(response.status, response.headers.get('content-type'));

const links = await response.json();
console.log(links.map((link) => link.title).join(', '));
console.log(new URL('links.json', location.href).pathname);
```

Output

```
200 application/json; charset=utf-8
MDN, Node.js
/examples/10-fetch-local/links.json
```

আপেক্ষিক ঠিকানা `'links.json'` Page-এর ঠিকানার সঙ্গে মিলিয়ে পূর্ণ হয়েছে (শেষ লাইন), ঠিক `new URL(relative, base)`-এর মতো। চলমান Request বাতিল এবং সময়সীমার জন্য অধ্যায় ১০-এর `AbortController` ও `AbortSignal.timeout` একইভাবে প্রযোজ্য।

১৫.৬ History API: অবস্থা URL-এ

একটি খোঁজার Page-এ ব্যবহারকারী "javascript" খুঁজলেন, তারপর Link-টি সহকর্মীকে পাঠালেন। সহকর্মী Link খুলে খোঁজার ফল না পেয়ে ফাঁকা Page পেলে অভিজ্ঞতা খারাপ। সমাধান: Page-এর গুরুত্বপূর্ণ অবস্থা URL-এ রাখা। **History API** Page Reload না করেই Address Bar-এর URL বদলায়।

history.js · JavaScript চলবে: Browser

```
const params = new URLSearchParams(location.search);
params.set('q', 'javascript');
history.replaceState(null, '', `${location.pathname}?${params}`);
console.log(location.search);

history.pushState({ page: 2 }, '', '?q=javascript&page=2');
console.log(location.search, history.state.page);

window.addEventListener('popstate', (event) => {
  console.log(`Back: ${location.search}, state ${JSON.stringify(event.state)}`);
});
history.back();
```

Output

```
?q=javascript
?q=javascript&page=2 2
Back: ?q=javascript, state null
```

Method	কাজ	Back Button
`history.replaceState(state, '', url)`	বর্তমান URL বদলায়	নতুন ধাপ তৈরি হয় না
`history.pushState(state, '', url)`	নতুন URL যোগ করে	Back চাপলে আগেরটিতে ফেরে
`popstate` Event	Back বা Forward চাপলে	URL ও `state` থেকে Page আবার আঁকুন

খোঁজার প্রতিটি অক্ষরে `pushState` করলে ব্যবহারকারীকে Back বিশবার চাপতে হবে; তাই লেখার সময় `replaceState`, আর "নতুন Page"-এর মতো বড় পরিবর্তনে `pushState`। Project খোঁজা ও ছাঁকনি `replaceState` দিয়ে URL-এ রাখে।

১৫.৭ Blob, File এবং Download

Blob হলো কাঁচা তথ্যের একটি টুকরো (লেখা বা বাইনারি), একটি ধরনসহ। **File** একটি Blob, যার সঙ্গে নাম ও তারিখ আছে; ব্যবহারকারী `<input type="file">` দিয়ে যে File দেন, তা এই ধরনের।

blob.mjs · JavaScript চলবে: Browser Console ও Node.js

```
const blob = new Blob(['নাম,শহর\n', 'Sara,Dhaka\n'], { type: 'text/csv' });
console.log(blob.type, blob.size);
console.log((await blob.text()).split('\n')[1]);

const file = new File(['{"ok":true}'], 'result.json', {
  type: 'application/json',
});
console.log(file.name, file.size, JSON.parse(await file.text()).ok);
```

Output

```
text/csv 31
Sara,Dhaka
result.json 11 true
```

`size` গোনে Byte, অক্ষর নয়: প্রতিটি বাংলা অক্ষর UTF-8-এ তিন Byte, তাই "নাম,শহর" অংশ ১৯ Byte (অধ্যায় ৮)। `text()` একটি Promise ফেরত দেয়, কারণ বড় File পড়তে সময় লাগে।

Browser-এ একটি Blob ব্যবহারকারীর কম্পিউটারে নামাতে:

download.js · JavaScript চলবে: Browser

```
const text = JSON.stringify({ hello: 'world' });
const blob = new Blob([text], { type: 'application/json' });
const link = document.createElement('a');

link.href = URL.createObjectURL(blob);
link.download = 'data.json';
link.click();

setTimeout(() => URL.revokeObjectURL(link.href), 1000);
```

`URL.createObjectURL(blob)` Memory-তে থাকা Blob-এর জন্য একটি সাময়িক `blob:` ঠিকানা তৈরি করে; `download` Attribute-সহ একটি Link-এ Click করলে Browser File হিসেবে নামায়। কাজ শেষে `revokeObjectURL` সেই Memory মুক্ত করে। এই উদাহরণটি Automated Run-এ চালানো হয়নি, কারণ এটি একটি File নামায়; Project-এর Browser Test একই Code-এর তৈরি Blob-এর বিষয়বস্তু আসল Chrome-এ যাচাই করে।

ব্যবহারকারীর File পড়া: `<input type="file">`-এর `change` Event-এ `input.files[0]` একটি File, এবং `await file.text()` তার বিষয়বস্তু। পুরোনো Code-এ একই কাজ `FileReader` দিয়ে দেখবেন। ব্যবহারকারীর File সম্পূর্ণ অবিশ্বস্ত: যেকোনো আকার, যেকোনো বিষয়বস্তু। আকার সীমা, JSON পার্সের ত্রুটি এবং গঠন—তিনটিই যাচাই করুন।

১৫.৮ নিরাপত্তা ও গোপনীয়তা

- **Web Storage Encrypt করা নয়,** এবং একই Website-এর যেকোনো Script পড়তে পারে। একটি XSS দুর্বলতা (অধ্যায় ৮) থাকলে আক্রমণকারীর Script সব পড়ে নেয়। Password, Login Token বা ব্যক্তিগত সংবেদনশীল তথ্য সেখানে রাখবেন না; Login-এর জন্য Server-এর `HttpOnly` Cookie ব্যবহৃত হয় (অধ্যায় ২০)।
- **ব্যবহারকারীর দেওয়া URL** Protocol যাচাই ছাড়া Link বা `src`-এ নয়। বাইরের Link-এ `rel="noopener noreferrer"`, যাতে নতুন Tab আপনার Page নিয়ন্ত্রণ করতে না পারে।
- **URL-এ গোপন তথ্য নয়।** URL Browser History, Server Log এবং শেয়ার করা Link-এ থেকে যায়।
- **Storage যেকোনো সময় মুছে যেতে পারে:** ব্যবহারকারী মুছতে পারেন, Browser জায়গা বাঁচাতে মুছতে পারে। গুরুত্বপূর্ণ তথ্যের জন্য Export-এর সুবিধা দিন, বা Server-এ রাখুন।
- **ব্যবহারকারীর তথ্য তাঁর নিয়ন্ত্রণে:** কী রাখা হচ্ছে স্পষ্ট করুন, এবং মোছার উপায় দিন।

Browser-এ তথ্য রাখার আরেকটি পুরোনো ব্যবস্থা **Cookie**। পার্থক্যটি জানা জরুরি, কারণ ভুল জায়গায় ভুল তথ্য রাখা নিরাপত্তার সাধারণ উৎস:

	Cookie	Web Storage
Server-এ যায় কি	প্রতিটি Request-এর সঙ্গে নিজে থেকে	কখনো না; JavaScript নিজে পাঠালে তবেই
আকার	প্রতিটি প্রায় ৪ KB	প্রায় ৫ MB
JavaScript থেকে লুকানো যায় কি	হ্যাঁ, `HttpOnly` দিলে	না
সাধারণ ব্যবহার	Login Session (Server নির্ধারণ করে)	Client-এর পছন্দ ও সাময়িক তথ্য

Login-এর পরিচয় তাই Server-এর দেওয়া `HttpOnly` Cookie-তে থাকে, যা কোনো Script—আক্রমণকারীর Script-সহ—পড়তে পারে না; আর অ-সংবেদনশীল Client-তথ্য থাকে Web Storage-এ।

১৫.৯ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
String জুড়ে URL বানানো	`&`, `#`, বাংলা অক্ষরে ভাঙা URL	`URL`, `URLSearchParams`
URL-এর মান যাচাই না করা	`NaN` Page, ভুল অবস্থা	রূপান্তর, যাচাই, Default
`javascript:` URL Link-এ	Click-এ Code চলে	শুধু `http:` / `https:`
`setItem`-এ `try` নেই	Storage ভর্তি হলে App থামে	`try` / `catch` ও বার্তা
Object সরাসরি Storage-এ	`"[object Object]"`	`JSON.stringify`
সংরক্ষিত তথ্যে Version নেই	App হালনাগাদের পর পুরোনো তথ্য অপাঠ্য	`version` ও Migration

ভুল	কী দেখা যায়	সমাধান
নিজের Tab-এ `storage` Event আশা করা	কিছুই ঘটে না	নিজের পরিবর্তনে নিজে আঁকুন
প্রতিটি অক্ষরে `pushState`	Back Button বিরক্তিকর	`replaceState`
`createObjectURL`-এর পর `revoke` নেই	Memory জমে	`revokeObjectURL`
ব্যবহারকারীর File বিশ্বাস	ভাঙা JSON-এ App থামে	পার্স ও গঠন যাচাই

১৫.১০ Debugging নির্দেশিকা

- **Application Panel (DevTools):** Local Storage ও Session Storage-এর প্রতিটি Key-Value দেখা, বদলানো ও মোছা যায়। "Reload-এ তথ্য থাকে না" সমস্যায় প্রথম জায়গা। Storage ভর্তির পরিস্থিতি পরীক্ষা করতে এখানে "Storage" অংশে সীমা অনুকরণ করা যায়।
- **Network Panel:** প্রতিটি Request-এর Method, Header, পাঠানো Body (Payload) ও সাড়া। POST-এ `Content-Type` আছে কি না এখানে দেখুন।
- **দুটি Tab পাশাপাশি খুলে** `storage` Event পরীক্ষা করুন; একই Tab-এ পরীক্ষা করলে Event আসে না।
- **URL-এর অবস্থা:** Address Bar-এর URL নিজে বদলে Reload করুন; Page সেই অবস্থায় খোলে কি না দেখুন। Back ও Forward চেপে দেখুন।
- **Console-এ** `new URL(text)` লিখে ব্যবহারকারীর দেওয়া ঠিকানা কীভাবে পড়া হচ্ছে দেখুন।
- **Download কাজ না করলে** Console-এ ত্রুটি দেখুন, এবং `blob:` ঠিকানা আগেভাগে `revoke` হচ্ছে কি না।

১৫.১১ পেশাদার অভ্যাস

- **Browser API সীমানায়, নিয়ম Pure Module-এ;** তাহলে নিয়মগুলো Node.js-এ Test করা যায়।
- **প্রতিটি বাইরের উৎস Trust Boundary:** URL, Storage, File, Server—সবকিছু যাচাই।
- **Feature Detection,** Browser-এর নাম নয়।
- **Storage-এর তথ্যে Version,** এবং প্রতিটি লেখা `try`-এ।
- **শেয়ারযোগ্য অবস্থা URL-এ,** Tab-এর সাময়িক অবস্থা `sessionStorage`-এ, দীর্ঘস্থায়ী পছন্দ `localStorage`-এ।
- **ব্যবহারকারীর তথ্য তাঁর হাতে:** Export ও Import।
- **Test-এ আসল Browser** যেখানে API শুধু Browser-এর; Node.js-এর Test-কে Browser-এর যাচাই বলবেন না।

১৫.১২ Project: Bookmark Vault

ডেভেলপারদের প্রয়োজনীয় Link-এর একটি ব্যক্তিগত ভান্ডার: Link যোগ, Tag, প্রিয় চিহ্ন, খোঁজা ও ছাঁকা, এবং File-এ Export ও Import। এই অধ্যায়ের প্রতিটি API এখানে একটি বাস্তব কাজে ব্যবহৃত।

Project-এর চাহিদা

- শিরোনাম ১ থেকে ১০০ অক্ষর; URL শুধু `http:` বা `https:`, `#` অংশ বাদ; একই URL দুবার নয়।
- Tag কমা দিয়ে; ছোট হাতের অক্ষরে, পুনরাবৃত্তি বাদ।
- প্রিয় চিহ্ন, মোছা; খোঁজা (শিরোনাম ও URL), Tag এবং "শুধু প্রিয়" ছাঁকনি।
- খোঁজা ও ছাঁকনি URL-এ (`?q=&tag=&fav=1`); Link শেয়ার বা Reload-এ একই অবস্থা।
- তালিকা `localStorage` -এ; অন্য Tab-এ পরিবর্তন হলে এখানেও হালনাগাদ (`storage` Event)।
- অর্ধেক পূরণ করা Form `sessionStorage` -এ; Reload-এ ফিরে আসে, যোগ করার পর মুছে যায়।
- Export: `version` -সহ JSON File নামানো (Blob)। Import: JSON File পড়া; অবৈধ ও পুনরাবৃত্ত Bookmark বাদ, সংখ্যা জানানো; ভাঙা File-এ বার্তা।
- সব লেখা `textContent` ; বাইরের Link-এ `rel="noopener noreferrer"` ।

File-এর কাঠামো

Folder-এর কাঠামো

```
bookmark-vault/
├── index.html
├── style.css
├── app.mjs                 UI ও Browser API
├── src/
│   ├── bookmarks.mjs       নিয়ম, যাচাই, Import ও Export (Pure)
│   ├── url-state.mjs       ছাঁকনি থেকে URL এবং URL থেকে ছাঁকনি
│   └── storage.mjs         localStorage Adapter
├── README.md
└── tests/
    ├── bookmarks.test.mjs
    ├── url-state.test.mjs
    ├── static-server.mjs
    ├── cdp.mjs
    └── browser-check.mjs
```

src/bookmarks.mjs

দুই অংশ মিলে সম্পূর্ণ File।

bookmarks.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
export const SCHEMA_VERSION = 1;
const ALLOWED_PROTOCOLS = ['http:', 'https:'];
const MAX_TITLE = 100;

export class BookmarkError extends Error {
  constructor(message) {
    super(message);
    this.name = 'BookmarkError';
  }
}

// শুধু http/https; # অংশ বাদ; Host ছোট হাতের অক্ষরে (URL নিজে করে)।
export function normalizeUrl(text) {
  let url;
  try {
    url = new URL(String(text ?? '').trim());
  } catch {
    throw new BookmarkError('সঠিক একটি URL লিখুন (https://...)।');
  }
  if (!ALLOWED_PROTOCOLS.includes(url.protocol)) {
    throw new BookmarkError(`"${url.protocol}" ধরনের URL গ্রহণযোগ্য নয়।`);
  }
  url.hash = '';
  return url.href;
}

export function parseTags(input) {
  const parts = Array.isArray(input) ? input : String(input ?? '').split(',');
  const tags = parts
    .map((tag) => String(tag).trim().toLowerCase())
    .filter(Boolean);
  return [...new Set(tags)];
}

const defaults = { now: () => new Date(), makeId: () => crypto.randomUUID() };

export function createBookmark(input, { now, makeId } = defaults) {
  const title = String(input?.title ?? '').trim();
  if (title === '' || title.length > MAX_TITLE) {
    throw new BookmarkError(`শিরোনাম 1 থেকে ${MAX_TITLE} অক্ষরের হতে হবে।`);
  }
  return {
    id: makeId(),
    title,
    url: normalizeUrl(input.url),
    tags: parseTags(input.tags),
    favorite: false,
    createdAt: now().toISOString(),
```

```
  };
}

// ...
```

bookmarks.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

export function addBookmark(list, bookmark) {
  if (list.some((item) => item.url === bookmark.url)) {
    throw new BookmarkError('এই URL আগে থেকেই আছে।');
  }
  return [bookmark, ...list];
}

export const toggleFavorite = (list, id) =>
  list.map((item) =>
    (item.id === id ? { ...item, favorite: !item.favorite } : item));

export const removeBookmark = (list, id) => list.filter((item) => item.id !== id);

export function filterBookmarks(list, filters = {}) {
  const { query = '', tag = '', favoritesOnly = false } = filters;
  const needle = query.trim().toLowerCase();
  return list.filter((item) =>
    (!favoritesOnly || item.favorite) &&
    (tag === '' || item.tags.includes(tag)) &&
    `${item.title} ${item.url}`.toLowerCase().includes(needle));
}

export const allTags = (list) =>
  [...new Set(list.flatMap((item) => item.tags))].sort();

// বাইরে থেকে আসা Bookmark (Storage বা Import) যাচাই; ঠিক না হলে null।
export function reviveBookmark(raw) {
  const validShape = typeof raw?.id === 'string' && raw.id !== '' &&
    typeof raw.favorite === 'boolean' && !Number.isNaN(Date.parse(raw.createdAt));
  if (!validShape) {
    return null;
  }
  try {
    const clock = { now: () => new Date(raw.createdAt), makeId: () => raw.id };
    return { ...createBookmark(raw, clock), favorite: raw.favorite };
  } catch (error) {
    if (error instanceof BookmarkError) {
      return null;
    }
    throw error;
```

```
  }
}

export function parseStored(text) {
  try {
    const data = JSON.parse(text ?? '[]');
    return Array.isArray(data) ? data.map(reviveBookmark).filter(Boolean) : [];
  } catch {
    return [];
  }
}

export function exportData(list, now = new Date()) {
  const data = {
    version: SCHEMA_VERSION,
    exportedAt: now.toISOString(),
    bookmarks: list,
  };
  return JSON.stringify(data, null, 2);
}

export function importData(text, existing) {
  let data;
  try {
    data = JSON.parse(text);
  } catch {
    throw new BookmarkError('File-টি সঠিক JSON নয়।');
  }
  if (data?.version !== SCHEMA_VERSION || !Array.isArray(data.bookmarks)) {
    throw new BookmarkError('অচেনা File-এর গঠন বা Version।');
  }

  let list = existing;
  let added = 0;
  for (const raw of data.bookmarks) {
    const bookmark = reviveBookmark(raw);
    const duplicate = bookmark && list.some((item) =>
      item.url === bookmark.url || item.id === bookmark.id);
    if (bookmark && !duplicate) {
      list = [...list, bookmark];
      added += 1;
    }
  }
  return { list, added, skipped: data.bookmarks.length - added };
}
```

- **`normalizeUrl`** `URL`-কে যাচাইকারী হিসেবে ব্যবহার করে: অবৈধ লেখায় তার `TypeError`-কে ব্যবহারকারীর ভাষার `BookmarkError`-এ রূপান্তর, তারপর Protocol-এর অনুমোদিত তালিকা। `javascript:` বা `data:`

এখানেই আটকায়।

- `new Set(tags)` : `Set` এমন একটি সংগ্রহ, যেখানে প্রতিটি মান একবারই থাকে; Array থেকে Set, আবার Spread দিয়ে Array—পুনরাবৃত্তি বাদের প্রচলিত কৌশল।
- `createBookmark` **-এর সময় ও ID Inject করা** (অধ্যায় ১২, ১৪): Test-এ নির্দিষ্ট তারিখ ও ID দেওয়া যায়, আর `reviveBookmark` একই Function দিয়ে সংরক্ষিত Bookmark-কে তার নিজের তারিখ ও ID-সহ আবার যাচাই করে।
- `reviveBookmark` **সব বাইরের তথ্যের দরজা:** Storage ও Import দুটিই এখান দিয়ে যায়, তাই একটি `javascript:` URL হাতে বসানো Storage বা File দিয়েও ঢুকতে পারে না।
- `importData` **কখনো বিদ্যমান তালিকা বদলায় না;** নতুন তালিকা, যোগ ও বাদের সংখ্যা ফেরত দেয়।

src/url-state.mjs এবং src/storage.mjs

url-state.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
export function readFilters(search) {
  const params = new URLSearchParams(search);
  return {
    query: params.get('q') ?? '',
    tag: params.get('tag') ?? '',
    favoritesOnly: params.get('fav') === '1',
  };
}

export function filtersToSearch({ query, tag, favoritesOnly }) {
  const params = new URLSearchParams();
  if (query.trim() !== '') {
    params.set('q', query.trim());
  }
  if (tag !== '') {
    params.set('tag', tag);
  }
  if (favoritesOnly) {
    params.set('fav', '1');
  }
  const text = params.toString();
  return text === '' ? '' : `?${text}`;
}
```

storage.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { parseStored } from './bookmarks.mjs';

export const STORAGE_KEY = 'jsm-ch15-bookmarks';

export function createStore(
  storage = globalThis.localStorage,
```

```
  key = STORAGE_KEY,
) {
  return {
    load() {
      try {
        return parseStored(storage.getItem(key));
      } catch {
        return [];
      }
    },
    save(list) {
      try {
        storage.setItem(key, JSON.stringify(list));
        return true;
      } catch {
        return false;
      }
    },
  };
}
```

`url-state.mjs` URL-এর সঙ্গে কাজ করে কিন্তু `location` বা `history` ছোঁয় না: লেখা নেয়, লেখা দেয়। তাই Node.js-এ Test করা যায়; `history.replaceState` ডাকে শুধু `app.mjs`।

index.html ও style.css

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Bookmark Vault</title>
  <link rel="stylesheet" href="style.css">
  <script type="module" src="app.mjs"></script>
</head>
<body>
  <main class="app">
    <h1>Bookmark Vault</h1>

    <form id="bookmark-form">
      <label>শিরোনাম <input name="title" autocomplete="off"></label>
      <label>URL <input name="url" type="url" placeholder="https://"></label>
      <label>Tag (কমা দিয়ে) <input name="tags" autocomplete="off"></label>
      <button type="submit">যোগ করুন</button>
    </form>

    <div class="filters">
```

```
      <label>খুঁজুন <input id="search" type="search"></label>
      <label>Tag <select id="tag"></select></label>
      <label><input id="favorites-only" type="checkbox"> শুধু প্রিয়</label>
    </div>

    <p id="count"></p>
    <ul id="bookmarks"></ul>

    <div class="files">
      <button id="export" type="button">Export</button>
      <label>Import
        <input id="import" type="file" accept="application/json,.json">
      </label>
    </div>
    <p id="status" role="status" aria-live="polite"></p>
  </main>
</body>
</html>
```

style.css · CSS — চলবে: Browser

```
body {
  margin: 0;
  font-family: 'Hind Siliguri', 'Nirmala UI', sans-serif;
  background: #f4f6fa;
  color: #1c2430;
}

.app {
  max-width: 720px;
  margin: 32px auto;
  padding: 24px;
  background: #ffffff;
  border-radius: 12px;
}

form,
.filters,
.files,
#bookmarks li {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 8px;
}

#bookmarks li {
  padding: 8px 0;
  border-bottom: 1px solid #e3e7ef;
```

```
}

.tags {
  color: #4a5668;
  font-size: 0.9em;
}

[aria-pressed='true'] {
  background: #ffe9a8;
}
```

app.mjs

তিন অংশ মিলে সম্পূর্ণ File।

app.mjs · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Browser

```
import {
  addBookmark, allTags, BookmarkError, createBookmark, exportData,
  filterBookmarks, importData, removeBookmark, toggleFavorite,
} from './src/bookmarks.mjs';
import { createStore, STORAGE_KEY } from './src/storage.mjs';
import { filtersToSearch, readFilters } from './src/url-state.mjs';

const FORM_STORE_KEY = 'jsm-ch15-draft';
const store = createStore();
let bookmarks = store.load();
let filters = readFilters(location.search);

const $ = (selector) => document.querySelector(selector);
const form = $('#bookmark-form');
const list = $('#bookmarks');
const statusView = $('#status');
const search = $('#search');
const tagSelect = $('#tag');
const favoritesOnly = $('#favorites-only');

function setBookmarks(next, message) {
  bookmarks = next;
  const saved = store.save(bookmarks);
  statusView.textContent = saved
    ? message
    : 'সংরক্ষণ করা যায়নি; Page বন্ধ করলে পরিবর্তন হারাবে।';
  render();
}

// প্রত্যাশিত ত্রুটি (BookmarkError) বার্তা হয়; অন্য সব ত্রুটি ওপরে যায়।
function attempt(action) {
  try {
```

```
    action();
  } catch (error) {
    if (!(error instanceof BookmarkError)) {
      throw error;
    }
    statusView.textContent = error.message;
  }
}

// ...
```

app.mjs · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Browser

```
// ...

function button(label, action) {
  const element = document.createElement('button');
  element.type = 'button';
  element.textContent = label;
  element.dataset.action = action;
  return element;
}

function bookmarkRow(item) {
  const link = document.createElement('a');
  link.href = item.url;
  link.textContent = item.title;
  link.target = '_blank';
  link.rel = 'noopener noreferrer';

  const tags = document.createElement('span');
  tags.className = 'tags';
  tags.textContent = item.tags.join(', ');

  const star = button('প্রিয়', 'favorite');
  star.setAttribute('aria-pressed', String(item.favorite));

  const row = document.createElement('li');
  row.dataset.id = item.id;
  row.append(link, tags, star, button('মুছুন', 'remove'));
  return row;
}

function renderTagOptions() {
  const tags = allTags(bookmarks);
  if (!tags.includes(filters.tag)) {
    filters = { ...filters, tag: '' };
  }
  tagSelect.replaceChildren(...['', ...tags].map((tag) => {
```

```
    const option = document.createElement('option');
    option.value = tag;
    option.textContent = tag === '' ? 'সব Tag' : tag;
    return option;
  }));
  tagSelect.value = filters.tag;
}

function render() {
  renderTagOptions();
  if (search.value !== filters.query) {
    search.value = filters.query;
  }
  favoritesOnly.checked = filters.favoritesOnly;

  const shown = filterBookmarks(bookmarks, filters);
  list.replaceChildren(...shown.map(bookmarkRow));
  $('#count').textContent = `${bookmarks.length}টির মধ্যে ${shown.length}টি দেখানো হচ্ছে`;
  const address = `${location.pathname}${filtersToSearch(filters)}`;
  history.replaceState(null, '', address);
}

// ...
```

app.mjs · JavaScript · অংশ ৩ / ৩ (Part 3 of 3) চলবে: Browser

```
// ...

form.addEventListener('submit', (event) => {
  event.preventDefault();
  attempt(() => {
    const bookmark = createBookmark(Object.fromEntries(new FormData(form)));
    setBookmarks(addBookmark(bookmarks, bookmark), `যোগ হয়েছে: ${bookmark.title}`);
    form.reset();
    sessionStorage.removeItem(FORM_STORE_KEY);
  });
});

// খসড়া শুধু এই Tab-এর; Reload-এর পরও থাকে।
form.addEventListener('input', () => {
  const draft = Object.fromEntries(new FormData(form));
  sessionStorage.setItem(FORM_STORE_KEY, JSON.stringify(draft));
});

list.addEventListener('click', (event) => {
  const action = event.target.closest('[data-action]')?.dataset.action;
  const id = event.target.closest('li')?.dataset.id;
  if (action === 'favorite') {
    setBookmarks(toggleFavorite(bookmarks, id), 'প্রিয় তালিকা বদলেছে।');
```

```
  }
  if (action === 'remove') {
    setBookmarks(removeBookmark(bookmarks, id), 'মুছে ফেলা হয়েছে।');
  }
});

const updateFilters = (change) => {
  filters = { ...filters, ...change };
  render();
};
search.addEventListener('input', () => updateFilters({ query: search.value }));
tagSelect.addEventListener('change', () =>
  updateFilters({ tag: tagSelect.value }));
favoritesOnly.addEventListener('change', () =>
  updateFilters({ favoritesOnly: favoritesOnly.checked }));

$('#export').addEventListener('click', () => {
  const blob = new Blob([exportData(bookmarks)], { type: 'application/json' });
  const link = document.createElement('a');
  link.href = URL.createObjectURL(blob);
  link.download = 'bookmarks.json';
  link.click();
  setTimeout(() => URL.revokeObjectURL(link.href), 1000);
  statusView.textContent = `${bookmarks.length}টি Bookmark Export হয়েছে।`;
});

$('#import').addEventListener('change', async (event) => {
  const [file] = event.target.files;
  if (!file) {
    return;
  }
  statusView.textContent = 'পড়া হচ্ছে...';
  try {
    const text = await file.text();
    const { list: next, added, skipped } = importData(text, bookmarks);
    setBookmarks(next, `Import: ${added}টি যোগ, ${skipped}টি বাদ।`);
  } catch (error) {
    if (!(error instanceof BookmarkError)) {
      throw error;
    }
    statusView.textContent = error.message;
  } finally {
    event.target.value = '';
  }
});

// অন্য Tab-এ তালিকা বদলালে এখানেও।
window.addEventListener('storage', (event) => {
  if (event.key === STORAGE_KEY) {
    bookmarks = store.load();
```

```
    render();
    statusView.textContent = 'অন্য Tab-এ তালিকা বদলেছে।';
  }
});

try {
  const draft = JSON.parse(sessionStorage.getItem(FORM_STORE_KEY) ?? '{}');
  for (const [name, value] of Object.entries(draft)) {
    if (form.elements[name]) {
      form.elements[name].value = value;
    }
  }
} catch {
  sessionStorage.removeItem(FORM_STORE_KEY);
}
render();
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **শুরু:** তালিকা `localStorage` থেকে (যাচাইসহ), ছাঁকনি URL থেকে (`readFilters(location.search)`), এবং অর্ধেক লেখা Form `sessionStorage` থেকে। ভাঙা খসড়া হলে চুপচাপ মুছে ফেলা হয়।
- `render` **URL হালনাগাদ করে** `history.replaceState` দিয়ে; তাই প্রতিটি খোঁজা বা ছাঁকনি URL-এ প্রতিফলিত হয়, কিন্তু Back Button-এর ইতিহাস ভরে না। Tag-টি তালিকায় আর না থাকলে (যেমন শেষ Bookmark মোছার পর) ছাঁকনি নিজে থেকে "সব Tag"-এ ফেরে।
- **Export:** Pure `exportData` লেখা তৈরি করে; Blob, `createObjectURL`, Link-এ Click এবং পরে `revokeObjectURL` —অনুচ্ছেদ ১৫.৭-এর ধাপগুলো।
- **Import:** `await file.text()`, তারপর Pure `importData`, যা ভাঙা JSON বা অচেনা Version-এ `BookmarkError` দেয়। `finally`-তে File-এর ঘর খালি করা হয়, যাতে একই File আবার বাছলেও `change` Event ঘটে। পড়ার আগে "পড়া হচ্ছে…" বার্তা বড় File-এ ব্যবহারকারীকে আশ্বস্ত করে।
- `storage` **Event:** অন্য Tab তালিকা বদলালে এই Tab আবার `store.load()` করে আঁকে। নিজের পরিবর্তনে এই Event আসে না, তাই `setBookmarks` নিজেই আঁকে।
- **নিরাপত্তা:** সব লেখা `textContent`; `href`-এ বসার আগে প্রতিটি URL `normalizeUrl`-এর Protocol যাচাই পেরিয়েছে; বাইরের Link নতুন Tab-এ `noopener`-সহ।

চালানো এবং প্রত্যাশিত আচরণ

Project Folder থেকে Local Server (Module ও `crypto.randomUUID`-এর জন্য):

Terminal

```
python -m http.server 8000
```

Browser-এ `http://localhost:8000` । Test:

Terminal

```
node --test
node tests/browser-check.mjs
```

কাজ	যা হওয়ার কথা
MDN, `https://developer.mozilla.org/#top`, Tag `JS, Docs, js`	URL-এ `#top` নেই; Tag `js, docs`
URL `javascript:alert(1)`	"\"javascript:\" ধরনের URL গ্রহণযোগ্য নয়।"
একই URL আবার	"এই URL আগে থেকেই আছে।"
খুঁজুন `node`, Tag `docs`, শুধু প্রিয়	URL হয় `?q=node&tag=docs&fav=1`; Reload-এ একই ছাঁকনি
Form অর্ধেক লিখে Reload	লেখা ফিরে আসে; যোগ করার পর মুছে যায়
Export, তারপর অন্য Browser-এ Import	একই Bookmark; পুনরাবৃত্তগুলো "বাদ"
ভাঙা JSON Import	"File-টি সঠিক JSON নয়।"
দুটি Tab খোলা, একটিতে যোগ	অন্যটিতে তালিকা ও "অন্য Tab-এ তালিকা বদলেছে।"

Companion Source Code-এর Node.js Test (২১টি) নিয়ম, Import-Export ও URL-এর রূপান্তর পরীক্ষা করে। `browser-check.mjs` একটি Local Server চালিয়ে আসল Chrome-এ ওপরের প্রতিটি সারি যাচাই করে: Export-এর Blob-এর বিষয়বস্তু পড়ে দেখে, Import-এ DevTools Protocol দিয়ে আসল File বাছাই করে, এবং একটি `<iframe>`-কে দ্বিতীয় Tab হিসেবে ব্যবহার করে `storage` Event পরীক্ষা করে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Page ফাঁকা, Console-এ CORS	`file://` থেকে খোলা	Local Server
`crypto.randomUUID is not a function`	Secure Context নয় (যেমন LAN-এর IP)	`localhost` বা `https://`
Reload-এ ছাঁকনি হারায়	`replaceState` ডাকা হয়নি	`render`-এর শেষ লাইন
অন্য Tab হালনাগাদ হয় না	`storage` Listener নেই, বা Key ভিন্ন	`STORAGE_KEY`
একই File দ্বিতীয়বার Import হয় না	File-এর ঘর খালি করা হয়নি	`event.target.value = ''`
Import-এ সব বাদ	Export-এর Version ভিন্ন	`SCHEMA_VERSION` ও Migration

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য শুধু এই Browser-এ; দুটি Tab একসঙ্গে লিখলে শেষ লেখাটি টেকে। Import-এর File-এর আকার সীমিত করা হয়নি; বড় File-এর আগে `file.size` যাচাই যোগ করা উচিত। শুধু একটি Schema Version আছে, তাই Migration-এর ধাপ এখনো দরকার হয়নি।

১৫.১৩ অনুশীলনী

প্রাথমিক স্তর

১. `https://example.com/search?q=dom&page=4` থেকে `hostname`, `q` এবং `page` (সংখ্যা হিসেবে) বের করুন।

২. `{ city: 'Dhaka', type: 'hotel & resort' }` থেকে `URLSearchParams` দিয়ে Query String তৈরি করুন।

৩. `localStorage`-এ একটি সংখ্যা রেখে আবার পড়ুন, এবং দেখান যে সেটি লেখা হয়ে গেছে; সঠিকভাবে সংখ্যায় ফেরান।

৪. নিচের প্রতিটির জন্য `localStorage`, `sessionStorage` না URL—কোনটি উপযুক্ত? (ক) Dark Mode পছন্দ (খ) এই Tab-এর অর্ধেক লেখা মন্তব্য (গ) খোঁজার শব্দ, যা Link-এ শেয়ার হবে।

৫. `isSafeLink(text)` লিখুন: শুধু `https:` URL-এ `true`; অবৈধ লেখায় `false`, ত্রুটি নয়।

মধ্যম স্তর

৬. অনুচ্ছেদ ১৫.৪-এর `migrate`-এ Version ৩ যোগ করুন: প্রতিটি কাজে `createdAt: null`।

৭. `readPage(search)` লিখুন: `?page=` থেকে ১ বা বেশি পূর্ণসংখ্যা; না থাকলে বা অবৈধ হলে ১।

৮. শিক্ষার্থীদের তালিকা থেকে CSV লেখার একটি Blob তৈরি করুন এবং তার আকার ও প্রথম সারি দেখান।

৯. একটি নকল Storage দিয়ে দেখান Project-এর `createStore`-এর `load` ভাঙা তথ্যে ফাঁকা তালিকা দেয়।

চ্যালেঞ্জ স্তর

১০. `filtersToSearch` ও `readFilters` একে অপরের উল্টো কি না পরীক্ষা করুন: একটি ছাঁকনি URL-এ নিয়ে আবার পড়লে একই ছাঁকনি পাওয়া যায় কি?

১১. Project-এর `exportData` ও `importData` দিয়ে Round-Trip দেখান: Export করে খালি তালিকায় Import, তারপর একই File আবার Import।

১২. `safeSave(key, value)` লিখুন, যা `QuotaExceededError` ধরে ব্যবহারকারীর ভাষায় বার্তা দেয়; আসল Browser-এ পরীক্ষা করুন।

১৫.১৪ অনুশীলনীর সমাধান

সমাধান ১

read-url.js · JavaScript — চলবে: Browser Console ও Node.js

```
const url = new URL('https://example.com/search?q=dom&page=4');
const page = Number(url.searchParams.get('page'));

console.log(url.hostname, url.searchParams.get('q'), page + 1);
```

Output

```
example.com dom 5
```

`page + 1` দেখায় সংখ্যায় রূপান্তর কাজ করেছে; রূপান্তর ছাড়া ফল হতো `'41'`।

সমাধান ২

build-query.js · JavaScript — চলবে: Browser Console ও Node.js

```
const params = new URLSearchParams({ city: 'Dhaka', type: 'hotel & resort' });

console.log(`https://example.com/search?${params}`);
```

Output

```
https://example.com/search?city=Dhaka&type=hotel+%26+resort
```

সমাধান ৩

number-storage.js · JavaScript — চলবে: Browser

```
localStorage.setItem('visits', 41);

const raw = localStorage.getItem('visits');
const visits = Number(raw) + 1;

console.log(typeof raw, raw + 1, visits);
localStorage.removeItem('visits');
```

Output

```
string 411 42
```

সমাধান ৪ — (ক) `localStorage`: দীর্ঘস্থায়ী পছন্দ, সব Tab-এ। (খ) `sessionStorage`: শুধু এই Tab, Reload-এ টেকে, Tab বন্ধ হলে যায়। (গ) URL: শেয়ার করা Link-এ অবস্থা থাকতে হবে; Storage অন্যের Browser-এ পৌঁছায় না।

সমাধান ৫

safe-link.js · JavaScript — চলবে: Browser Console ও Node.js

```
function isSafeLink(text) {
  try {
    return new URL(text).protocol === 'https:';
  } catch {
    return false;
  }
}

console.log(isSafeLink('https://example.com'), isSafeLink('http://example.com'));
console.log(isSafeLink('javascript:alert(1)'), isSafeLink('example.com'));
```

Output

```
true false
false false
```

`example.com`-এ Protocol নেই, তাই `new URL` ত্রুটি দেয়, এবং Function শান্তভাবে `false` ফেরত দেয়।

সমাধান ৬

migrate-v3.js · JavaScript — চলবে: Browser Console ও Node.js

```
function migrate(data) {
  let current = data;
  if (current?.version === 1) {
    const items = current.items.map((name) => ({ name, done: false }));
    current = { version: 2, items };
  }
  if (current?.version === 2) {
    const items = current.items.map((item) => ({ ...item, createdAt: null }));
    current = { version: 3, items };
  }
  if (current?.version !== 3) {
    throw new Error(`অচেনা Version: ${data?.version}`);
  }
  return current;
}
```

```
const fromV1 = migrate({ version: 1, items: ['বাজার'] });
console.log(fromV1.version, JSON.stringify(fromV1.items[0]));
```

Output

```
3 {"name":"বাজার","done":false,"createdAt":null}
```

ধাপগুলো পরপর চলে, তাই Version ১-এর তথ্যও দুটি ধাপ পেরিয়ে ৩-এ পৌঁছায়। নতুন Version মানে শুধু একটি নতুন ধাপ।

সমাধান ৭

read-page.js · JavaScript — চলবে: Browser Console ও Node.js

```
function readPage(search) {
  const page = Number(new URLSearchParams(search).get('page'));
  return Number.isInteger(page) && page >= 1 ? page : 1;
}

console.log(readPage('?page=3'), readPage('?page=0'), readPage('?page=2.5'));
console.log(readPage('?page=abc'), readPage(''));
```

Output

```
3 1 1
1 1
```

সমাধান ৮

csv-blob.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const students = [
  { name: 'Sara', marks: 88 },
  { name: 'Rafi', marks: 72 },
];

const lines = ['name,marks', ...students.map((s) => `${s.name},${s.marks}`)];
const blob = new Blob([lines.join('\n')], { type: 'text/csv' });

console.log(blob.size, (await blob.text()).split('\n')[1]);
```

Output

```
26 Sara,88
```

সমাধান ৯

broken-store.mjs · JavaScript চলবে: Node.js

```
import { createStore } from '../../project/bookmark-vault/src/storage.mjs';

const fakeStorage = {
  getItem: () => '{this is not json',
  setItem: () => {},
};

const store = createStore(fakeStorage);
console.log(store.load().length, store.save([]));
```

Output

```
0 true
```

সমাধান ১০

round-trip-url.mjs · JavaScript চলবে: Node.js

```
import {
  filtersToSearch, readFilters,
} from '../../project/bookmark-vault/src/url-state.mjs';

const cases = [
  { query: 'node & npm', tag: 'docs', favoritesOnly: true },
  { query: '', tag: '', favoritesOnly: false },
  { query: 'বাংলা', tag: '', favoritesOnly: false },
];

for (const filters of cases) {
  const search = filtersToSearch(filters);
  const back = readFilters(search);
  console.log(JSON.stringify(back) === JSON.stringify(filters), search || '(ফাঁকা)');
}
```

Output

```
true ?q=node+%26+npm&tag=docs&fav=1
true (ফাঁকা)
true ?q=%E0%A6%AC%E0%A6%BE%E0%A6%82%E0%A6%B2%E0%A6%BE
```

তিনটি ক্ষেত্রেই ফিরে পাওয়া ছাঁকনি মূলটির সমান। একটি সূক্ষ্মতা: আগে-পরের ফাঁকা জায়গাসহ খোঁজার শব্দ (`' node '`) ফিরে আসবে `'node'` হয়ে, কারণ `filtersToSearch` `trim` করে। Project-এর Test এই আচরণও যাচাই করে।

সমাধান ১১

round-trip-file.mjs · JavaScript — চলবে: Node.js

```
import {
  createBookmark, exportData, importData,
} from '../../project/bookmark-vault/src/bookmarks.mjs';

const clock = { now: () => new Date('2026-10-15T09:00:00Z'), makeId: () => 'b1' };
const input = { title: 'MDN', url: 'https://developer.mozilla.org/' };
const bookmark = createBookmark(input, clock);

const file = exportData([bookmark], new Date('2026-10-15T10:00:00Z'));
const first = importData(file, []);
const second = importData(file, first.list);

console.log(first.added, first.skipped, first.list[0].title);
console.log(second.added, second.skipped, second.list.length);
```

Output

```
1 0 MDN
0 1 1
```

দ্বিতীয়বার একই File: পুনরাবৃত্ত, তাই বাদ, তালিকা অপরিবর্তিত।

সমাধান ১২

safe-save.js · JavaScript — চলবে: Browser

```
function safeSave(key, value) {
  try {
    localStorage.setItem(key, value);
    return 'সংরক্ষিত';
  } catch (error) {
    if (error.name === 'QuotaExceededError') {
      return 'জায়গা শেষ; পুরোনো তথ্য মুছে আবার চেষ্টা করুন।';
    }
    return 'সংরক্ষণ করা যায়নি।';
  }
}

console.log(safeSave('small', 'ok'));
console.log(safeSave('big', 'x'.repeat(11 * 1024 * 1024)));
localStorage.removeItem('small');
```

Output

```
সংরক্ষিত
জায়গা শেষ; পুরোনো তথ্য মুছে আবার চেষ্টা করুন।
```

১৫.১৫ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. কোনটি Browser-এর API, JavaScript ভাষার অংশ নয়?

ক) `Array.prototype.map`

খ) `JSON.parse`

গ) `localStorage`

ঘ) `Promise`

প্রশ্ন ২. `new URL('not a url')` কী করে?

ক) `null` ফেরত দেয়

খ) `TypeError` ছোড়ে

গ) একটি খালি URL দেয়

ঘ) `about:blank` দেয়

প্রশ্ন ৩. `sessionStorage`-এর তথ্য কখন মুছে যায়?

ক) Page Reload করলে

খ) কখনো না

গ) Tab বন্ধ করলে

ঘ) প্রতি ২৪ ঘণ্টায়

প্রশ্ন ৪. `localStorage.setItem` জায়গা শেষ হলে কী করে?

ক) `QuotaExceededError` ছোড়ে

খ) পুরোনো তথ্য মুছে জায়গা করে

গ) নীরবে কিছু না করে

ঘ) `false` ফেরত দেয়

প্রশ্ন ৫. `storage` Event কোন Page পায়?

ক) যে Page পরিবর্তন করেছে

খ) একই Website-এর অন্য খোলা Page

গ) সব Website-এর সব Page

ঘ) শুধু Server

প্রশ্ন ৬. খোঁজার শব্দ লেখার সময় URL-এ রাখতে কোনটি উপযুক্ত, যাতে Back Button-এর ইতিহাস না ভরে?

ক) `history.pushState`

খ) `location.href = ...`

গ) `history.replaceState`

ঘ) `location.reload()`

প্রশ্ন ৭. ব্যবহারকারীর দেওয়া URL Link-এ বসানোর আগে কী যাচাই সবচেয়ে জরুরি?

ক) URL-এর দৈর্ঘ্য

খ) Protocol (`http:` / `https:`)

গ) Domain-এর বানান

ঘ) Port নম্বর

প্রশ্ন ৮. `new Blob(['বই']).size` কত?

ক) 6

খ) 2

গ) 1

ঘ) 4

প্রশ্ন ৯. সংরক্ষিত তথ্যে `version` রাখার উদ্দেশ্য?

ক) তথ্য Encrypt করা

খ) Storage-এর সীমা বাড়ানো

গ) App হালনাগাদের পর পুরোনো গঠনের তথ্য চিনে রূপান্তর করা

ঘ) Tab-এর সংখ্যা গোনা

প্রশ্ন ১০. Login Token রাখার জন্য `localStorage` কেন ঝুঁকিপূর্ণ?

ক) এটি ধীর

খ) এটি শুধু সংখ্যা রাখে

গ) Tab বন্ধ হলে মুছে যায়

ঘ) একই Website-এর যেকোনো Script, XSS-এর Script-সহ, পড়তে পারে

১৫.১৬ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–খ · ৩–গ · ৪–ক · ৫–খ · ৬–গ · ৭–খ · ৮–ক · ৯–গ · ১০–ঘ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	Storage Browser দেয়; Node.js-এ নেই। বাকিগুলো ভাষার অংশ।
২	খ	অবৈধ লেখায় `URL` Constructor `TypeError` দেয়।
৩	গ	Reload-এ টেকে; Tab বন্ধ হলে যায়।
৪	ক	ত্রুটি ছোড়ে, কিছুই সংরক্ষিত হয় না; তাই `try` বাধ্যতামূলক।
৫	খ	পরিবর্তনকারী Page নিজে পায় না।
৬	গ	বর্তমান ইতিহাসের ধাপ বদলায়, নতুন ধাপ যোগ করে না।
৭	খ	`javascript:` URL Click-এ Code চালায়।
৮	ক	UTF-8-এ প্রতিটি বাংলা অক্ষর ৩ Byte।
৯	গ	পুরোনো Version চিনে Migration-এর ধাপ চালানো যায়।
১০	ঘ	Storage Encrypt করা নয় এবং Script-এর নাগালে।

১৫.১৭ অধ্যায়ের সারসংক্ষেপ

- Browser API (DOM, Storage, Location, History) ভাষার বাইরে; `URL`, `fetch`, `Blob` Browser ও Node.js দুটিতেই আছে। Feature Detection করুন।
- `URL` ও `URLSearchParams` ঠিকানা পড়ে, গড়ে, Encode করে ও যাচাই করে; URL-এর মানও Trust Boundary।
- `localStorage` দীর্ঘস্থায়ী ও সব Tab-এ, `sessionStorage` Tab-এর আয়ু পর্যন্ত; দুটিই লেখা রাখে, সীমিত, এবং ব্যর্থ হতে পারে।
- `storage` Event অন্য Tab-কে পরিবর্তন জানায়।
- সংরক্ষিত ও Export-এর তথ্যে `version` এবং পর্যায়ক্রমিক Migration।
- `Request` Method, Header ও Body জমা রাখে; JSON পাঠাতে `Content-Type`।
- History API অবস্থা URL-এ রাখে: লেখায় `replaceState`, নতুন ধাপে `pushState`, Back-এ `popstate`।
- `Blob` ও `File` তথ্যের টুকরো; `createObjectURL` দিয়ে Download, `file.text()` দিয়ে পড়া; ব্যবহারকারীর File যাচাই।
- Web Storage-এ গোপন তথ্য নয়; ব্যবহারকারীর URL-এ Protocol যাচাই।

পরের অধ্যায়ে Software Reliability: Testing-এর ধরন, Node.js Test Runner-এর গভীর ব্যবহার, Test Double, এবং নির্ভরযোগ্য Program তৈরির পদ্ধতি।

১৫.১৮ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Browser API	Browser-এর দেওয়া সুবিধা, ভাষার অংশ নয়।
Feature Detection	ব্যবহারের আগে API আছে কি না যাচাই।
Secure Context	`https://` বা `localhost`; কিছু API শুধু এখানে।
URLSearchParams	URL-এর Query অংশ পড়া ও লেখার Object।
`sessionStorage`	Tab-এর আয়ু পর্যন্ত টেকা Storage।
Quota	প্রতি Website-এর Storage-সীমা।
`storage` Event	অন্য Tab-এ Storage বদলালে আসা Event।
Migration	পুরোনো গঠনের তথ্য নতুন গঠনে রূপান্তর।
History API	Reload ছাড়া URL বদলানো ও Back সামলানো।
Blob	কাঁচা তথ্যের টুকরো, একটি ধরনসহ।
Object URL	Memory-র Blob-এর জন্য সাময়িক `blob:` ঠিকানা।
`Set`	প্রতিটি মান একবারই থাকে এমন সংগ্রহ।

১৫.১৯ আরও পড়ুন

- **MDN — URL:** `https://developer.mozilla.org/en-US/docs/Web/API/URL` — `URL` ও `URLSearchParams`।
- **MDN — Web Storage API:** `https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API` — `localStorage`, `sessionStorage` ও `storage` Event।
- **MDN — Storage quotas and eviction criteria:** `https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria` — Storage-এর সীমা ও মুছে যাওয়া।
- **MDN — History API:** `https://developer.mozilla.org/en-US/docs/Web/API/History_API` — `pushState`, `replaceState`, `popstate`।
- **MDN — Blob:** `https://developer.mozilla.org/en-US/docs/Web/API/Blob` — Blob, File ও Object URL।